THE C PROGRAMMING LANGUAGE

Complete Notes — Basics to Advanced

A structured reference covering syntax, semantics, memory management, data structures, and best practices in C

Table of Contents

1. Introduction to C

1.1 What is C?

C: a general-purpose, procedural, compiled programming language developed by Dennis Ritchie at Bell Labs between 1969 and 1973. It was originally created to rewrite the UNIX operating system.

C is called a 'middle-level language' because it combines the features of a high-level language (readable syntax, portability) with the capabilities of a low-level language (direct memory access, bit manipulation, pointer arithmetic). This unique position is why C is still used today for operating systems, embedded systems, device drivers, and performance-critical applications.

1.2 Key Characteristics of C

- Procedural language: Programs are organized as a sequence of instructions and functions, executed step by step.
- Compiled language: Source code is translated into machine code by a compiler before execution, producing fast executables.
- Statically typed: The data type of every variable must be known at compile time.
- Low-level memory access: C allows direct manipulation of memory through pointers.
- Portable: A program written in standard C can be compiled and run on different machines with little or no modification.
- Small core language: C has a small set of keywords (32 in ANSI C); most functionality comes from libraries.
- Foundation language: C influenced C++, Java, C#, JavaScript, Python, Go, Rust, and many others.

1.3 Structure of a C Program

Every C program follows a fixed skeletal structure. Understanding this structure is the first step to reading or writing any C program.

```
#include <stdio.h>      // Preprocessor directive (header inclusion)
#define MAX 100         // Preprocessor directive (macro definition)

// Global declarations (variables, function prototypes)
int globalVar = 10;
void greet(void);       // function prototype

int main(void)          // Entry point of every C program
{
   int localVar = 5;   // Local variable declaration
   printf("Hello, World!\n");  // Statement
   greet();            // Function call
   return 0;           // Return statement (0 = success)
}
```

```
void greet(void)        // Function definition
{
   printf("Welcome to C programming.\n");
}
```

Explanation of each part

Preprocessor directives: lines beginning with # that are processed before compilation begins. #include brings in declarations from header files; #define creates macros.

Global declarations: variables and function prototypes declared outside any function, visible throughout the file.

main() function: the mandatory entry point. Execution of every C program always begins in main().

Statements: individual instructions, each terminated with a semicolon (;).

Comments: text ignored by the compiler, used for documentation. // for single-line, /* */ for multi-line.

return statement: sends a value back to the operating system (via main) or to the calling function, indicating how execution concluded.

1.4 The Compilation Process

C is a compiled language, meaning the source code goes through several distinct phases before it becomes a runnable program:

1. Preprocessing: The preprocessor expands macros, includes header files, and removes comments. Output: expanded source code.
2. Compilation: The compiler translates the preprocessed code into assembly language specific to the target processor.
3. Assembly: The assembler converts assembly code into machine code (object code), producing a .o or .obj file.
4. Linking: The linker combines object files with library code (e.g., the C standard library) and resolves references to produce a single executable file.

> ***Note:*** *You can see these stages manually with GCC: gcc -E (preprocess), gcc -S (compile to assembly), gcc -c (assemble to object file), gcc (full build with linking).*

1.5 Compiling and Running a C Program

Using the GNU Compiler Collection (GCC), a typical workflow looks like this:

```
gcc program.c -o program     // Compile program.c into an executable named 'program'
./program                    // Run the executable (Linux/macOS)
program.exe                  // Run the executable (Windows)
```

2. Variables, Data Types, and Constants

2.1 Variables

Variable: a named location in memory used to store a value that can change during program execution. Every variable in C must be declared with a specific data type before it is used.

Syntax: data_type variable_name; or data_type variable_name = initial_value;

```
int age = 21;        // declaration with initialization
float price;         // declaration only
price = 99.99;       // assignment afterward
int x, y, z = 0;     // multiple declarations
```

Rules for naming variables (identifiers)

- Must begin with a letter (a-z, A-Z) or an underscore (_), never a digit.
- Can contain letters, digits, and underscores only — no spaces or special characters.
- Case-sensitive: 'Age' and 'age' are different identifiers.
- Cannot be a reserved keyword (e.g., int, return, for).
- Convention: use meaningful, lowercase names with underscores or camelCase (total_marks or totalMarks).

2.2 Data Types

Data type: a classification that specifies what kind of value a variable can hold, how much memory it occupies, and what operations can be performed on it.

2.2.1 Primary (Basic) Data Types

Type	Typical Size	Description	Format Specifier
int	4 bytes	Whole numbers (signed by default)	%d
float	4 bytes	Single-precision floating-point number	%f
double	8 bytes	Double-precision floating-point number	%lf
char	1 byte	A single character (stored as its ASCII value)	%c
void	0 bytes	Represents 'no value' — used for functions returning nothing, or generic pointers	—

> ***Note:*** *Sizes are compiler/platform dependent. Use the sizeof operator to check the exact size on your system, e.g., printf("%zu", sizeof(int));*

2.2.2 Type Modifiers

Modifiers adjust the size or sign behaviour of the basic types:

- signed — allows both negative and positive values (default for int).
- unsigned — allows only zero and positive values, doubling the positive range.
- short — reduces the storage size (short int, usually 2 bytes).
- long — increases the storage size (long int, long long int, usually 4/8 bytes).

2.2.3 Derived and User-Defined Data Types (overview — detailed later)

- Arrays — collection of elements of the same type.
- Pointers — variables that store memory addresses.
- Structures (struct) — group different data types under one name.
- Unions (union) — like structures, but members share the same memory.
- Enumerations (enum) — user-defined set of named integer constants.

2.3 Constants

Constant: a value that cannot be changed during program execution.

```
const float PI = 3.14159;      // 'const' qualifier — compiler enforced
#define MAX_SIZE 100           // macro constant — replaced by preprocessor before compilation
```

Difference between the two: const creates a real, typed variable that the compiler protects from modification and which occupies memory; #define performs simple textual substitution with no type checking and no memory allocation of its own.

2.4 Type Conversion

Implicit type conversion (coercion): automatic conversion performed by the compiler when operands of different types are mixed in an expression, following promotion rules (e.g., int automatically becomes float when used with a float).

Explicit type conversion (type casting): manual conversion specified by the programmer using the cast operator.

```
int a = 7, b = 2;
float result = (float)a / b;   // Explicit cast: result = 3.500000
float bad    = a / b;          // Without cast: integer division gives 3, then stored as 3.000000
```

2.5 Input and Output

C's standard library (stdio.h) provides functions for console input/output.

Function	Purpose
printf()	Prints formatted output to the console.
scanf()	Reads formatted input from the keyboard.

getchar() / putchar()	Reads/writes a single character.
gets() / fgets()	Reads a line of text (gets() is unsafe and deprecated; fgets() is the safe alternative).
puts()	Writes a string followed by a newline.

```
#include <stdio.h>

int main(void)
{
   int age;
   char name[50];

   printf("Enter your name: ");
   scanf("%s", name);          // %s reads a single word (no spaces)

   printf("Enter your age: ");
   scanf("%d", &age);          // & gives the address where input is stored

   printf("Hello %s, you are %d years old.\n", name, age);
   return 0;
}
```

> ***Note:*** *scanf() requires the address-of operator (&) for normal variables because it needs to know WHERE to store the value, not just its current value. Arrays (like 'name' above) already decay to an address, so & is not used with them.*

Common format specifiers

Specifier	Used For
%d / %i	int
%f	float
%lf	double
%c	char
%s	string (char array)
%x / %o	hexadecimal / octal integer
%u	unsigned int
%p	pointer address
%%	literal percent sign

3. Operators and Expressions

Operator: a symbol that instructs the compiler to perform a specific mathematical, relational, or logical operation on one or more operands.

3.1 Arithmetic Operators

Operator	Meaning	Example (a=10, b=3)
+	Addition	a + b = 13
-	Subtraction	a - b = 7
*	Multiplication	a * b = 30
/	Division	a / b = 3 (integer division truncates)
%	Modulus (remainder)	a % b = 1

3.2 Relational Operators

Relational operators compare two values and produce a result of 1 (true) or 0 (false).

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

3.3 Logical Operators

Operator	Meaning	Example
&&	Logical AND — true only if both operands are true	(a>0 && b>0)
||	Logical OR — true if at least one operand is true	(a>0 || b>0)
!	Logical NOT — reverses the truth value	!(a>0)

> ***Note:*** *C uses 'short-circuit evaluation' for && and ||: in (a && b), if a is false, b is never evaluated, because the result is already determined.*

3.4 Bitwise Operators

Bitwise operators work directly on the binary representation of integer values — a hallmark of C's low-level capability.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR (exclusive OR)
~	Bitwise NOT (one's complement)
<<	Left shift (multiplies by 2 per shift)
>>	Right shift (divides by 2 per shift)

```
unsigned char a = 5;   // binary: 0000 0101
unsigned char b = 9;   // binary: 0000 1001
printf("%d\n", a & b);   // 0000 0001 -> 1
printf("%d\n", a | b);   // 0000 1101 -> 13
printf("%d\n", a ^ b);   // 0000 1100 -> 12
printf("%d\n", a << 1);  // 0000 1010 -> 10
printf("%d\n", a >> 1);  // 0000 0010 -> 2
```

3.5 Assignment Operators

Operator	Equivalent To
=	a = b
+=	a = a + b
-=	a = a - b
*=	a = a * b
/=	a = a / b
%=	a = a % b

3.6 Increment and Decrement Operators

++ (increment) and -- (decrement): unary operators that increase or decrease a variable's value by 1.

Pre-increment (++a) increases the value first, then uses it in the expression. Post-increment (a++) uses the current value first, then increases it.

```
int a = 5;
printf("%d\n", ++a);  // prints 6 (a becomes 6, then used)
printf("%d\n", a++);  // prints 6 (used first, THEN becomes 7)
printf("%d\n", a);    // prints 7
```

3.7 Conditional (Ternary) Operator

Ternary operator (?:): the only operator in C that takes three operands; a shorthand for a simple if-else.

```
int a = 10, b = 20, max;
max = (a > b) ? a : b;   // equivalent to: if (a > b) max = a; else max = b;
printf("%d\n", max);   // 20
```

3.8 sizeof Operator

sizeof: a compile-time operator that returns the number of bytes occupied by a variable or data type.

```
printf("%zu\n", sizeof(int));     // typically 4
printf("%zu\n", sizeof(double));  // typically 8
```

3.9 Operator Precedence and Associativity

Precedence: determines which operator is evaluated first when an expression contains multiple operators.

Associativity: determines the order of evaluation when operators of the SAME precedence appear together (left-to-right or right-to-left).

Example: in the expression 10 + 5 * 2, multiplication (*) has higher precedence than addition (+), so it is evaluated first: 10 + (5*2) = 20, not (10+5)*2 = 30. When in doubt, use parentheses to make the intended order explicit and the code more readable.

4. Control Flow Statements

Control flow: the order in which individual statements, instructions, or function calls are executed. By default, C executes statements sequentially; control statements alter this default flow.

4.1 Decision-Making Statements

4.1.1 if statement

Executes a block of code only if a condition evaluates to true (non-zero).

```
if (marks >= 40) {
   printf("Pass\n");
}
```

4.1.2 if-else statement

Provides an alternative block of code to run when the condition is false.

```
if (marks >= 40) {
   printf("Pass\n");
} else {
   printf("Fail\n");
}
```

4.1.3 if-else if-else ladder

Used to test multiple conditions in sequence.

```
if (marks >= 90) {
   printf("Grade A\n");
} else if (marks >= 75) {
   printf("Grade B\n");
} else if (marks >= 40) {
   printf("Grade C\n");
} else {
   printf("Fail\n");
}
```

4.1.4 switch statement

switch: a multi-way branch statement that compares a single variable against several constant values, providing a cleaner alternative to a long if-else ladder when checking one variable against many discrete values.

```
switch (grade) {
   case 'A':
      printf("Excellent\n");
      break;          // 'break' prevents fall-through to the next case
   case 'B':
```

```
        printf("Good\n");
        break;
    default:            // executes if no case matches
        printf("Invalid grade\n");
}
```

> ***Note:*** *Without 'break', execution 'falls through' into the next case automatically. This is sometimes used intentionally (multiple cases sharing one action) but is a very common source of bugs when forgotten.*

4.2 Looping (Iteration) Statements

Loop: a control structure that repeats a block of code as long as a specified condition remains true.

4.2.1 for loop

Best suited when the number of iterations is known in advance. It combines initialization, condition, and update in one line.

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
// Output: 1 2 3 4 5
```

Structure: for (initialization; condition; update) — initialization runs once, condition is checked before every iteration, and update runs after every iteration.

4.2.2 while loop

An entry-controlled loop: the condition is checked BEFORE the loop body executes. If false initially, the body never runs.

```
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}
```

4.2.3 do-while loop

An exit-controlled loop: the body executes first, and the condition is checked AFTER. Guarantees at least one execution of the loop body.

```
int i = 1;
do {
    printf("%d ", i);
    i++;
} while (i <= 5);
```

4.2.4 Nested loops

A loop inside another loop, commonly used for processing two-dimensional data such as matrices.

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        printf("%d ", i * j);
    }
    printf("\n");
}
```

4.3 Jump Statements

Statement	Effect
break	Immediately terminates the nearest enclosing loop or switch statement.
continue	Skips the rest of the current iteration and moves to the next iteration of the loop.
goto	Transfers control unconditionally to a labeled statement elsewhere in the function (rarely used; considered poor practice in modern C).
return	Exits the current function, optionally returning a value to the caller.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) break;       // loop stops entirely when i is 5
    if (i % 2 == 0) continue; // skips printing even numbers
    printf("%d ", i);
}
// Output: 1 3
```

5. Functions

Function: a self-contained, named block of code designed to perform a specific task. Functions allow large programs to be broken into smaller, reusable, and manageable pieces — a principle known as modular programming.

5.1 Why Use Functions?

- Reusability — write once, call many times.
- Modularity — break complex problems into smaller sub-problems.
- Readability and maintainability — easier to debug and update isolated pieces of logic.
- Abstraction — the caller only needs to know WHAT a function does, not HOW it does it.

5.2 Function Declaration, Definition, and Call

Function prototype (declaration): tells the compiler the function's name, return type, and parameter types in advance, before its actual body appears. Required when a function is called before it is defined in the file.

Function definition: contains the actual body — the statements that execute when the function is called.

Function call: the statement that invokes (executes) the function, optionally passing arguments and receiving a return value.

```
#include <stdio.h>

int add(int a, int b);     // 1. Prototype (declaration)

int main(void) {
   int sum = add(3, 4);   // 3. Function call (arguments 3 and 4)
   printf("%d\n", sum);  // Output: 7
   return 0;
}

int add(int a, int b) {    // 2. Function definition
   return a + b;          // 'a' and 'b' here are called parameters
}
```

> ***Note:*** *Terminology distinction: 'parameters' are the variable names listed in the function definition; 'arguments' are the actual values passed during the function call.*

5.3 Parameter Passing Methods

5.3.1 Call by Value

Call by value: a copy of the actual argument's value is passed to the function. Changes made to the parameter inside the function do NOT affect the original variable in the caller.

```
void modify(int x) {
```

```
    x = x + 10;          // only changes the local copy
}
int main(void) {
    int num = 5;
    modify(num);
    printf("%d\n", num); // still prints 5 — original is unchanged
    return 0;
}
```

5.3.2 Call by Reference (using pointers)

Call by reference: the memory address of the actual argument is passed (via a pointer), allowing the function to directly modify the original variable.

```
void modify(int *x) {
    *x = *x + 10;        // dereference and change the ORIGINAL value
}
int main(void) {
    int num = 5;
    modify(&num);        // pass the address of num
    printf("%d\n", num); // prints 15 — original IS changed
    return 0;
}
```

> ***Note:*** *C does not have true 'pass by reference' syntax like C++; it simulates the effect by explicitly passing pointers (addresses).*

5.4 Return Values

A function can return at most one value directly, using the return statement. Its type must match the function's declared return type. A function declared with return type void returns no value.

```
int square(int n) {
    return n * n;     // returns a single integer
}
```

5.5 Recursion

Recursion: a technique where a function calls itself, either directly or indirectly, to solve a problem by breaking it into smaller instances of the same problem.

Every recursive function needs: (1) a base case that stops the recursion, and (2) a recursive case that moves the problem toward the base case. Without a base case, recursion continues indefinitely, eventually causing a stack overflow.

```
int factorial(int n) {
    if (n == 0) return 1;          // base case
    return n * factorial(n - 1);   // recursive case
}
// factorial(4) = 4 * factorial(3) = 4 * 3 * factorial(2) = ... = 24
```

5.6 Storage Classes

Storage class: specifies the scope (visibility), lifetime, and default initial value of a variable or function.

Storage Class	Scope	Lifetime	Description
auto	Local (block)	Until block ends	Default storage class for local variables; rarely written explicitly.
static	Local or file	Entire program run	Retains its value between function calls; a static local variable is initialized only once.
extern	Global (whole program)	Entire program run	Declares a variable/function defined in another file, enabling sharing across multiple source files.
register	Local (block)	Until block ends	Suggests the compiler store the variable in a CPU register for faster access (modern compilers largely ignore this and optimize automatically).

```
void counter(void) {
    static int count = 0;   // initialized only ONCE, across all calls
    count++;
    printf("%d\n", count);
}
// Calling counter() three times prints: 1  2  3  (not 1 1 1)
```

5.7 Scope of Variables

Local variable: declared inside a function or block; accessible only within that function or block.

Global variable: declared outside all functions; accessible from any function in the file (and other files, if declared extern).

> ***Note:*** *Excessive use of global variables is generally discouraged in good C programming practice because it makes programs harder to debug and reason about — any function can silently modify shared state.*

6. Arrays and Strings

6.1 Arrays

Array: a collection of elements of the SAME data type, stored in contiguous (back-to-back) memory locations, and accessed using a single name combined with an index.

Indexing starts at 0: in an array of size n, valid indices run from 0 to n-1.

6.1.1 One-Dimensional Arrays

```
int marks[5] = {90, 85, 78, 92, 88};  // declaration + initialization
printf("%d\n", marks[0]);            // 90 (first element)
printf("%d\n", marks[4]);            // 88 (last element)
marks[2] = 80;                       // modifying an element

// Iterating over an array
for (int i = 0; i < 5; i++) {
    printf("%d ", marks[i]);
}
```

> ***Note:*** *C performs NO automatic bounds checking. Accessing marks[10] on a 5-element array does not raise an error — it reads/writes adjacent (garbage) memory, leading to undefined behaviour. This is a common source of bugs called a 'buffer overflow'.*

6.1.2 Multi-Dimensional Arrays

Two-dimensional array: essentially an array of arrays, often used to represent matrices or tables of data.

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
printf("%d\n", matrix[1][2]);   // 6 (row index 1, column index 2)

for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}
```

6.1.3 Arrays and Memory

An array's name represents the address of its first element. This is why arrays are said to 'decay' into a pointer when passed to a function — the function receives the starting address, not a copy of the whole array.

6.2 Strings

String: in C, a sequence of characters stored in a character array, terminated by a special null character '\0' (ASCII value 0) that marks the end of the string.

```
char name[6] = "Hello";      // stored as: 'H','e','l','l','o','\0'
                             // note: needs 6 bytes for a 5-letter word
char name2[] = "World";      // size automatically calculated as 6
printf("%s\n", name);        // %s prints until it finds '\0'
```

> ***Note:*** *Because strings rely on the null terminator, a string array must always be large enough to hold all characters PLUS the '\0'. Forgetting this is a classic source of buffer overflows.*

6.2.1 Common String Functions (string.h)

Function	Purpose
strlen(s)	Returns the length of string s, NOT counting the null terminator.
strcpy(dest, src)	Copies src into dest, including the null terminator.
strcat(dest, src)	Appends (concatenates) src onto the end of dest.
strcmp(s1, s2)	Compares two strings lexicographically; returns 0 if equal, negative if s1<s2, positive if s1>s2.
strchr(s, c)	Finds the first occurrence of character c in string s.
strstr(s1, s2)	Finds the first occurrence of substring s2 within s1.

```
#include <string.h>
char a[20] = "Hello";
char b[20] = "World";

printf("%lu\n", strlen(a));     // 5
strcat(a, b);                   // a becomes "HelloWorld"
printf("%d\n", strcmp(a, b));   // non-zero, since a != b
```

> ***Note:*** *strcpy and strcat do not check buffer sizes. Modern, safer code often uses strncpy/strncat (which take a maximum length) to avoid overflowing the destination buffer.*

7. Pointers

Pointers are widely considered the most powerful — and most challenging — feature of C. Mastery of pointers is essential for understanding memory management, dynamic data structures, and how C interacts directly with hardware.

7.1 What is a Pointer?

Pointer: a variable that stores the memory address of another variable, rather than storing a data value directly.

```
int num = 50;       // a normal variable holding a value
int *ptr = &num;    // 'ptr' is a pointer holding the ADDRESS of num

printf("%d\n", num);    // 50          -- the value
printf("%p\n", &num);   // e.g. 0x7ffee  -- the address of num
printf("%p\n", ptr);    // e.g. 0x7ffee  -- same address, stored in ptr
printf("%d\n", *ptr);   // 50          -- dereferencing: 'the value AT this address'
```

Key symbols

Symbol	Name	Meaning
&	Address-of operator	Returns the memory address of a variable.
*	Dereference operator (when used on a pointer)	Accesses the value stored AT the address the pointer holds.
*	Pointer declarator (when used in a declaration)	Declares a variable as a pointer, e.g., int *p;

7.2 Pointer Declaration and Initialization

```
int *p;         // declares p as a pointer to an int (uninitialized — dangerous to use yet)
p = NULL;       // safe practice: explicitly set to NULL until a real address is assigned
int x = 10;
p = &x;         // now p correctly points to x
```

> ***Note:*** *A pointer's TYPE (int*, float*, char*, etc.) tells the compiler how many bytes to read/write at the address, and how much to move when doing pointer arithmetic. It does not change the address itself.*

7.3 Pointer Arithmetic

Because array elements are stored contiguously, adding 1 to a pointer moves it forward by exactly sizeof(type) bytes — not by 1 byte.

```
int arr[3] = {10, 20, 30};
```

```
int *p = arr;          // points to arr[0] (array name decays to address of first element)

printf("%d\n", *p);     // 10
printf("%d\n", *(p+1)); // 20  (moved forward by sizeof(int) bytes)
printf("%d\n", *(p+2)); // 30
p++;                    // now p points to arr[1]
```

7.4 Pointers and Arrays

arr[i] and *(arr + i) are exactly equivalent in C — array subscripting is internally implemented using pointer arithmetic.

7.5 Pointers to Pointers

Pointer to pointer (double pointer): a pointer that stores the address of another pointer, declared with two asterisks.

```
int x = 5;
int *p = &x;
int **pp = &p;     // pp holds the address of p

printf("%d\n", **pp);  // 5 -- double dereference reaches x's value
```

7.6 Null Pointers and Dangling Pointers

Null pointer: a pointer explicitly assigned the value NULL (typically 0), indicating it points to nothing valid. Always check for NULL before dereferencing a pointer that might not have been assigned.

Dangling pointer: a pointer that still holds the address of memory that has already been freed or has gone out of scope. Dereferencing a dangling pointer causes undefined behaviour.

```
int *p = malloc(sizeof(int));
free(p);          // memory is released
// p is now a DANGLING pointer -- it still holds the old address
*p = 10;          // UNDEFINED BEHAVIOUR -- writing to freed memory
p = NULL;         // good practice: nullify immediately after freeing
```

7.7 Void Pointers

Void pointer (void*): a generic pointer that can point to any data type, but cannot be dereferenced directly — it must first be cast to a specific type.

Void pointers are widely used for generic functions, such as malloc() (which returns void*) and qsort().

7.8 Function Pointers

Function pointer: a pointer that stores the address of a function rather than a variable, allowing functions to be passed as arguments, stored in arrays, or chosen dynamically at runtime.

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

int main(void) {
    int (*operation)(int, int);   // declares a pointer to a function
                                  // taking two ints and returning an int
    operation = add;
    printf("%d\n", operation(5, 3));  // 8

    operation = sub;
    printf("%d\n", operation(5, 3));  // 2
    return 0;
}
```

> ***Note:*** *Function pointers are the mechanism behind callbacks in C (e.g., the comparator function passed to qsort()), and they form the conceptual basis for how higher-level languages implement first-class functions and virtual method tables.*

7.9 Pointers and const

Declaration	Meaning
const int *p	p can point elsewhere, but the value it points to cannot be changed through p.
int * const p	p always points to the same address, but the value at that address can be changed.
const int * const p	Neither the address p holds, nor the value it points to, can change.

8. Dynamic Memory Management

Dynamic memory allocation: the process of requesting memory from the operating system at runtime (rather than at compile time), allowing programs to size their data structures based on conditions only known while running.

8.1 Memory Layout of a C Program

- Stack — stores local variables and function call information; managed automatically, fast, but limited in size.
- Heap — a large pool of memory used for dynamic allocation; managed manually by the programmer using malloc/free.
- Data segment — stores global and static variables.
- Code (text) segment — stores the compiled machine instructions of the program itself.

8.2 The Four Key Functions (stdlib.h)

Function	Purpose
malloc(size)	Allocates 'size' bytes of uninitialized memory on the heap; returns a void* pointer to the start, or NULL if allocation fails.
calloc(n, size)	Allocates memory for 'n' elements of 'size' bytes each, AND initializes all bytes to zero.
realloc(ptr, newSize)	Resizes a previously allocated block, preserving its existing contents as far as possible.
free(ptr)	Releases previously allocated memory back to the system so it can be reused.

```
#include <stdlib.h>

int n = 5;
int *arr = (int*) malloc(n * sizeof(int));   // request space for 5 ints

if (arr == NULL) {                           // ALWAYS check for allocation failure
   printf("Memory allocation failed.\n");
   exit(1);
}

for (int i = 0; i < n; i++) {
   arr[i] = i * i;       // use it just like a normal array
}

arr = realloc(arr, 10 * sizeof(int));   // grow it to hold 10 ints

free(arr);              // release the memory
arr = NULL;                 // avoid leaving a dangling pointer
```

8.3 Memory Leaks

Memory leak: occurs when dynamically allocated memory is never freed, even though the program no longer needs it or has lost all pointers to it. Over time, repeated leaks can exhaust available memory and crash long-running programs.

```
void leak(void) {
   int *p = malloc(100 * sizeof(int));
   // ... function ends WITHOUT calling free(p) ...
   // the allocated memory is now unreachable and wasted for the rest of the run
}
```

> ***Note:*** *Rule of thumb: every malloc/calloc/realloc should have a corresponding free. Tools like Valgrind are commonly used to detect leaks during development.*

8.4 malloc vs calloc — Quick Comparison

Aspect	malloc	calloc
Initialization	Leaves memory uninitialized (garbage values)	Initializes all bytes to zero
Arguments	Single argument: total bytes	Two arguments: number of elements, size of each
Speed	Slightly faster (no zeroing step)	Slightly slower due to zero-initialization

9. Structures, Unions, and Enumerations

9.1 Structures (struct)

Structure: a user-defined data type that groups together variables of different data types under a single name, allowing related data to be treated as one unit.

```
struct Student {
    char name[50];
    int roll;
    float marks;
};                      // semicolon required after the closing brace

int main(void) {
    struct Student s1 = {"Ayan", 101, 92.5};

    printf("%s\n", s1.name);     // accessing members with the dot (.) operator
    printf("%d\n", s1.roll);
    printf("%.1f\n", s1.marks);
    return 0;
}
```

9.1.1 Structures and Pointers

When accessing structure members through a pointer to a structure, C provides the arrow operator (->) as shorthand for dereferencing followed by dot access.

```
struct Student *sp = &s1;
printf("%s\n", (*sp).name);   // explicit dereference, then dot
printf("%s\n", sp->name);     // equivalent, cleaner arrow syntax
```

9.1.2 Nested Structures and Arrays of Structures

```
struct Student classroom[30];     // an array of 30 Student structures
classroom[0].roll = 1;
strcpy(classroom[0].name, "Ayan");
```

9.1.3 typedef with Structures

typedef: a keyword used to create an alias (alternative name) for an existing data type, often used to simplify long or repetitive struct declarations.

```
typedef struct {
    int x, y;
} Point;            // 'Point' can now be used directly, without writing 'struct'

Point p1 = {10, 20};  // instead of: struct Point p1 = {10, 20};
```

9.2 Unions (union)

Union: similar to a structure in syntax, but all members SHARE the same memory location. The union's total size equals the size of its LARGEST member, and only one member can hold a valid value at any given time.

```
union Data {
   int i;
   float f;
   char str[20];
};
// sizeof(union Data) = 20 (the size of the largest member, str)
// whereas an equivalent struct would be sizeof(int)+sizeof(float)+20
```

> ***Note:*** *Unions are useful when you need to store different types of data in the same memory at different times (saving space), such as representing a single value that could be 'either an int or a float, but never both at once.'*

9.3 Enumerations (enum)

Enumeration: a user-defined data type consisting of a set of named integer constants, improving code readability compared to using plain magic numbers.

```
enum Day {SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY};
// By default, SUNDAY=0, MONDAY=1, TUESDAY=2, ... SATURDAY=6

enum Day today = WEDNESDAY;
printf("%d\n", today);   // 3
```

10. File Handling

File handling: the mechanism by which C programs read from and write to files stored on disk, allowing data to persist beyond a single program execution. C accesses files through the FILE pointer type defined in stdio.h.

10.1 Opening and Closing Files

```
#include <stdio.h>

FILE *fp = fopen("data.txt", "w");   // open (or create) a file for writing
if (fp == NULL) {
    printf("Error opening file.\n");
    return 1;
}
// ... read/write operations ...
fclose(fp);                          // ALWAYS close a file once finished
```

File modes

Mode	Meaning
"r"	Read — file must already exist.
"w"	Write — creates a new file, or overwrites/truncates an existing one.
"a"	Append — writes are added to the end of the file; creates the file if it doesn't exist.
"r+"	Read and write — file must already exist.
"rb" / "wb"	Same as r/w, but in binary mode (no text translation of newlines).

10.2 Reading and Writing

Function	Purpose
fprintf(fp, ...)	Writes formatted text to a file (like printf, but to a file).
fscanf(fp, ...)	Reads formatted text from a file (like scanf, but from a file).
fgets(buf, n, fp)	Reads up to n-1 characters or a full line from a file into buf.
fputs(str, fp)	Writes a string to a file.
fread() / fwrite()	Reads/writes raw binary data block-by-block.

```
FILE *fp = fopen("data.txt", "w");
fprintf(fp, "Name: %s, Age: %d\n", "Ayan", 22);
fclose(fp);
```

```
fp = fopen("data.txt", "r");
char line[100];
while (fgets(line, sizeof(line), fp) != NULL) {
   printf("%s", line);   // print each line read from the file
}
fclose(fp);
```

10.3 Checking End of File

EOF (End Of File): a special signal returned by reading functions to indicate that there is no more data left to read from the file.

> ***Note:*** *Always close every file you open with fclose(). Leaving files open can cause data loss (unwritten buffered data) and resource leaks, particularly in long-running programs.*

11. The Preprocessor

Preprocessor: a program that runs automatically before actual compilation, processing all lines beginning with the '#' symbol. It performs text substitution and file inclusion; it does not understand C syntax or types.

11.1 Macros (#define)

```
#define PI 3.14159                // simple constant macro
#define SQUARE(x) ((x) * (x))       // function-like macro

printf("%f\n", PI);
printf("%d\n", SQUARE(5));          // expands to ((5) * (5)) = 25
```

> ***Note:*** *Always wrap macro parameters and the entire macro body in parentheses. Without them, SQUARE(a+b) would expand to a+b*a+b instead of the intended ((a+b)*(a+b)), due to operator precedence.*

11.2 Conditional Compilation

Allows different parts of code to be included or excluded depending on conditions checked at compile time — commonly used for platform-specific code or debug builds.

```
#define DEBUG 1

#if DEBUG
   printf("Debug mode active\n");
#endif

#ifdef _WIN32
   // Windows-specific code
#elif __linux__
   // Linux-specific code
#endif
```

11.3 Header Guards

Header guard: a preprocessor pattern that prevents a header file's contents from being included more than once in the same translation unit, which would otherwise cause 'redefinition' compiler errors.

```
// myheader.h
#ifndef MYHEADER_H     // if MYHEADER_H is not yet defined...
#define MYHEADER_H     // ...define it now (so this only runs once)

void greet(void);

#endif
```

11.4 Other Useful Directives

Directive	Purpose
#include	Inserts the contents of another file (a header) at this point.
#undef	Removes a previous macro definition.
#pragma	Gives compiler-specific instructions (e.g., #pragma once, an alternative to header guards).
#error	Forces a compilation error with a custom message, often used to enforce build conditions.

12. Multi-File Programs and the Compilation Model

Real-world C projects are split across multiple .c (source) and .h (header) files for organization and faster incremental builds.

12.1 Typical Project Layout

```
math_utils.h    <- declarations (function prototypes) shared between files
math_utils.c    <- definitions (actual implementation)
main.c          <- uses the functions declared in math_utils.h
```

```
// math_utils.h
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int add(int a, int b);
#endif

// math_utils.c
#include "math_utils.h"
int add(int a, int b) { return a + b; }

// main.c
#include <stdio.h>
#include "math_utils.h"
int main(void) {
    printf("%d\n", add(2, 3));
    return 0;
}
```

```
gcc -c math_utils.c -o math_utils.o   // compile each .c file into an object file
gcc -c main.c -o main.o
gcc math_utils.o main.o -o program    // link object files into one executable
```

12.2 extern for Sharing Global Variables

```
// file1.c
int counter = 0;          // definition — allocates actual storage

// file2.c
extern int counter;       // declaration — refers to the SAME variable defined in file1.c
```

12.3 The Standard Library — Common Headers

Header	Provides
stdio.h	Standard input/output (printf, scanf, file functions).
stdlib.h	General utilities: memory allocation, conversions, exit(), rand().
string.h	String manipulation functions.
math.h	Mathematical functions (sqrt, pow, sin, log, etc.).
ctype.h	Character classification/conversion (isdigit, toupper, etc.).
time.h	Date and time functions.
limits.h / float.h	Defines the minimum/maximum representable values for integer/floating types.

13. Building Data Structures in C

C does not provide built-in data structures like lists or maps (unlike Python or Java). Instead, programmers build them manually using structures and pointers. This section covers the most fundamental example: the linked list — a frequent interview topic.

13.1 Linked List

Linked list: a linear data structure made of 'nodes', where each node stores a data value and a pointer to the next node in the sequence. Unlike arrays, linked lists do not require contiguous memory and can grow or shrink dynamically at runtime.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;     // pointer to the next node
};

// Insert a new node at the front of the list, return the new head
struct Node* insertFront(struct Node *head, int value) {
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = head;     // new node points to the old head
    return newNode;           // new node becomes the new head
}

// Print all values in the list
void printList(struct Node *head) {
    struct Node *curr = head;
    while (curr != NULL) {
        printf("%d -> ", curr->data);
        curr = curr->next;
    }
    printf("NULL\n");
}

int main(void) {
    struct Node *head = NULL;          // start with an empty list
    head = insertFront(head, 30);
    head = insertFront(head, 20);
    head = insertFront(head, 10);
    printList(head);                   // 10 -> 20 -> 30 -> NULL
    return 0;
}
```

> ***Note:*** *This same node + pointer pattern, extended further, underlies stacks, queues, trees, and graphs — all of which are typically implemented in C using structures and dynamic memory rather than language built-ins.*

13.2 Stack and Queue (conceptual overview)

Stack: a Last-In-First-Out (LIFO) structure: the most recently added element is the first one removed. Typically supports push (add) and pop (remove) operations, implementable using either an array or a linked list.

Queue: a First-In-First-Out (FIFO) structure: the first element added is the first one removed. Typically supports enqueue (add) and dequeue (remove) operations.

14. Common Errors and Best Practices

14.1 Common Beginner-to-Intermediate Mistakes

Mistake	Why It's a Problem
Using = instead of == in a condition	if (x = 5) assigns 5 to x (always true) instead of comparing — a classic, hard-to-spot bug.
Missing break in switch	Causes unintended fall-through into the next case.
Array index out of bounds	C does not check this; it silently corrupts adjacent memory.
Forgetting to free() allocated memory	Causes memory leaks in long-running programs.
Using a pointer before initializing it	Dereferencing an uninitialized ('wild') pointer leads to undefined behaviour and crashes.
Returning a pointer to a local variable	The local variable's memory is reclaimed once the function returns, leaving a dangling pointer.
Comparing strings with ==	Compares pointer addresses, not string content; use strcmp() instead.
Off-by-one errors in loops	Using <= instead of < (or vice versa) causes one extra or missing iteration.

14.2 Best Practices

- Always initialize variables before use; never assume a default value.
- Check the return value of malloc/calloc/fopen and other functions that can fail.
- Match every malloc with exactly one free, and set the pointer to NULL afterward.
- Use const for values and parameters that should not be modified.
- Prefer fgets()/strncpy() over gets()/strcpy() to avoid buffer overflows.
- Enable compiler warnings (gcc -Wall -Wextra) and treat them seriously — most bugs surface here first.
- Keep functions small and focused on a single task; pass data explicitly rather than relying on globals.
- Comment intent, not the obvious — explain WHY, not just WHAT the code does.

Closing Note

This document covers the complete journey of C — from program structure and basic syntax through to pointers, dynamic memory, file handling, and the foundations of building data structures by hand. The concepts here (especially pointers, memory management, and storage classes) are also the topics most frequently probed in technical interviews, since they reveal genuine understanding of how a program actually executes and uses memory, rather than just familiarity with syntax.